

# 세미나 4회차: **Arduino / 3D 재구성 기술** **비교** (NeRF, 전통 MVS, 3D Gaussian Splatting(3DGS))

육시우

2026-07-07



**KNU**

# 목차

- 1. Arduino
- 2. NeRF
- 3. MVS
- 4. 3D Gaussian Splatting(3DGS)

# 1. Arduino

- **Arduino**란 하드웨어와 소프트웨어를 모두 포함한 **오픈소스 마이크로컨트롤러 플랫폼**

| 구성 요소 | 설명                           |
|-------|------------------------------|
| 하드웨어  | Arduino Uno, Nano, Mega 등 보드 |
| 소프트웨어 | Arduino IDE (코드를 작성하고 업로드)   |
| 언어    | C/C++ 기반 스케치(Sketch)         |

- Arduino IDE 설치

🔧 공식 설치 경로

👉 <https://www.arduino.cc/en/software>

| 운영체제         | 설치 방법                                                         |
|--------------|---------------------------------------------------------------|
| Windows      | <code>.exe</code> 설치 파일 다운로드 후 실행                             |
| macOS        | <code>.zip</code> 파일 압축 해제 후 <code>Applications</code> 폴더로 이동 |
| Ubuntu/Linux | 아래 명령으로 설치                                                    |

```
sudo apt update  
sudo apt install arduino
```

# 1. Arduino

## - 보드 연결

1. Arduino 보드를 USB 케이블로 PC에 연결
2. IDE 상단 메뉴에서 다음을 설정

| 메뉴                   | 설정 값                              |
|----------------------|-----------------------------------|
| <b>Tools → Board</b> | Arduino Uno (또는 보유 중인 모델)         |
| <b>Tools → Port</b>  | 연결된 포트 선택 (예: COM3, /dev/ttyUSB0) |

연결이 완료되면 IDE 하단에 “포트 연결됨(Connected)” 메시지가 표시됩니다.

# 1. Arduino 예제

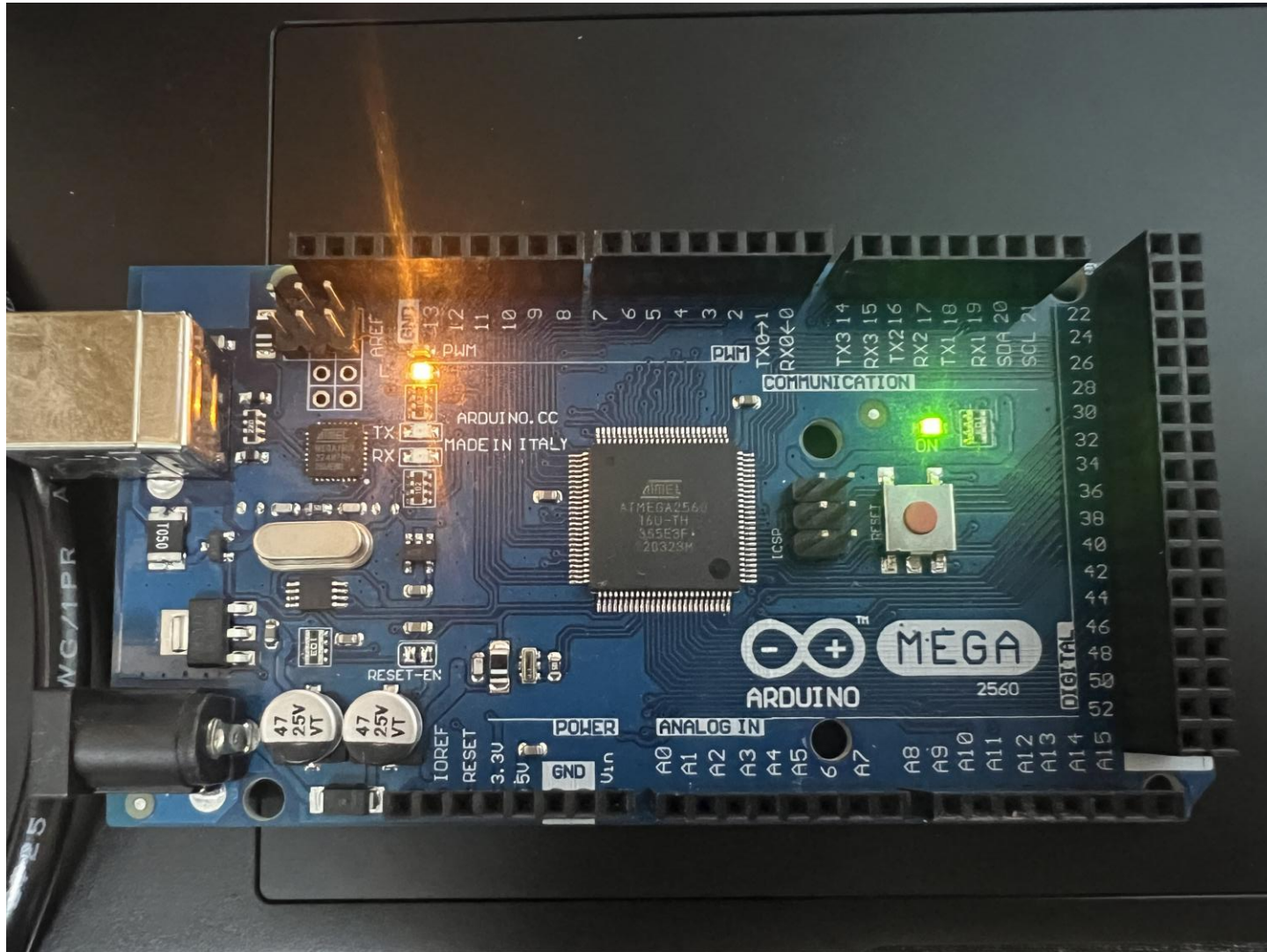
Blink 예제: 보드 내장 LED(D13 핀)를 1초 간격으로 켜고 끄는 코드

```
// Blink Example
void setup() {
    pinMode(13, OUTPUT); // 13번 핀을 출력 모드로 설정
}

void loop() {
    digitalWrite(13, HIGH); // LED 켜기
    delay(1000);             // 1초 대기
    digitalWrite(13, LOW);  // LED 끄기
    delay(1000);             // 1초 대기
}
```

## 코드 업로드 방법

- IDE 상단의  **Verify** 버튼으로 코드 컴파일
- → **Upload** 버튼으로 보드에 코드 업로드

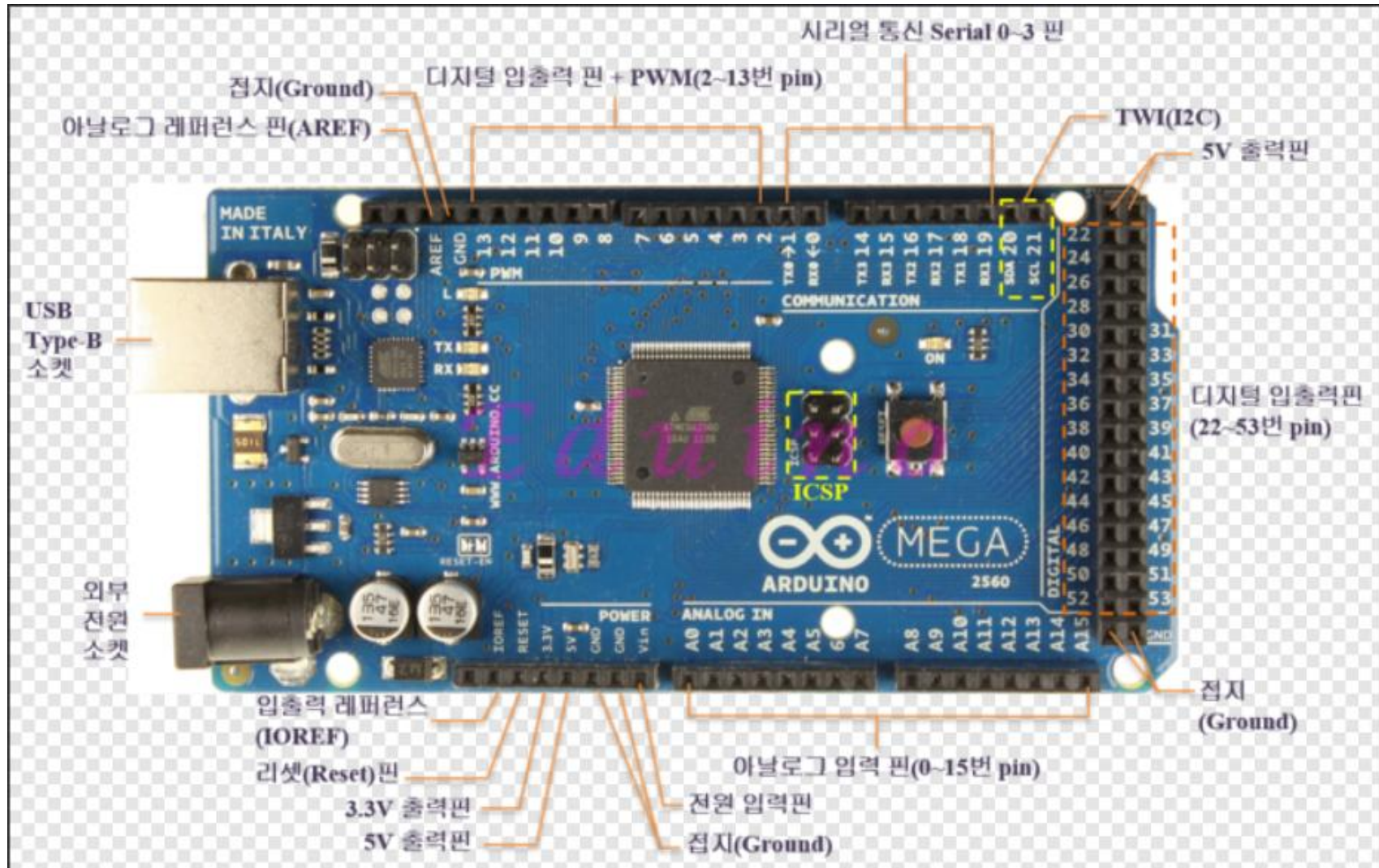


# 1. Arduino Mega 알아보기

| Microcontroller                               | ATmega2560                                         |
|-----------------------------------------------|----------------------------------------------------|
| 동작전압(Operating Voltage)                       | 5V                                                 |
| 정격전압(Input Voltage) (recommended)             | 7-12V                                              |
| 허용전압<br>(Input Voltage) (limits)              | 6-20V                                              |
| 디지털 입출력핀<br>(Digital I/O Pins)                | 54개 (이 중 15개는 PWM사용 가능)                            |
| 아날로그 입력핀<br>(Analog Input Pins)               | 16개                                                |
| DC 입출력핀의 출력 전류<br>(DC Current per I/O Pin)    | 40 mA                                              |
| DC 3.3v일 때 출력 전류<br>(DC Current for 3.3V Pin) | 50mA                                               |
| 플래시 메모리<br>(Flash Memory)                     | 256KB (ATmega2560) of which 8KB used by bootloader |
| SRAM                                          | 8KB                                                |
| EEPROM                                        | 4KB                                                |
| 클럭속도(Clock Speed)                             | 16MHz                                              |

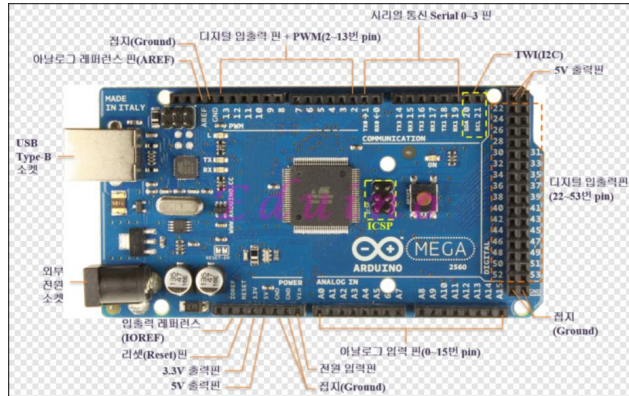


# 1. Arduino Mega 알아보기





# 1. Arduino Mega 알아보기



IOREF : I/O Reference Voltage(입출력 기준 전압)의 약자로, 보드가 현재 사용하는 디지털 입출력(I/O) 신호의 동작 전압이 몇 v인지를 외부 확장 보드(쉴드, Shield)에 알려주는 기준 핀이다. 아두이노에서 쉴드(Shield)란, 아두이노 보드 위에 레고 블록처럼 층층이 꽂아서 특정 기능을 바로 확장할 수 있도록 만든 전용 확장 보드이다. 따라서 우리는 거의 사용할 일이 없다.

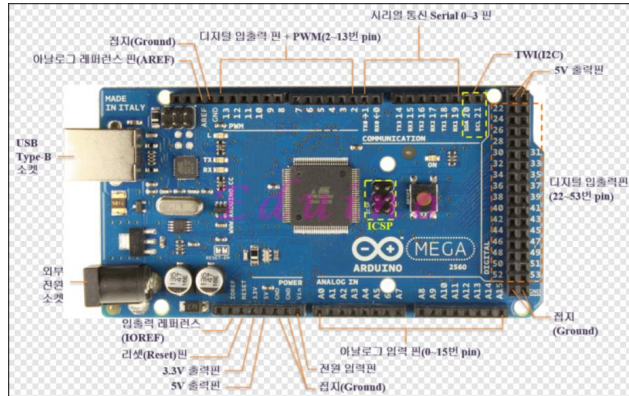
RESET : RESET(리셋) 핀과 리셋 버튼은 아두이노 메가의 마이크로컨트롤러를 초기화하여, 현재 실행 중인 코드를 강제로 처음부터(맨 첫 줄부터) 다시 시작하게 만드는 장치

3.3V / 5V 출력핀 : 아두이노 보드에서 외부 장치에게 전원을 공급할 때 사용

전원 입력핀(Vin) : 아두이노 보드를 컴퓨터와 연결하지 않고 단독으로 사용할 때 전원을 공급하기 위함

아날로그 입력 핀 16개

# 1. Arduino Mega 알아보기



디지털 입력 핀 54개

15개 핀 : PWM 가능. PWM(펄스 폭 변조) - 아두이노 같은 디지털 칩은 0V(LOW), 5V(HIGH) 딱 두 가지만 출력할 수 있어서 중간 전압을 출력할 수 없다. 이때 PWM을 사용하면 5V와 0V를 빠른 속도로 깜빡이게 만들어서, 평균 전압을 0V~5V 사이로 자유롭게 조절한다. → 듀티 사이클에 따라 전압이 결정됨.

시리얼 통신 핀(UART) : 시리얼 통신은 우리말로 직렬 통신이라고 부르며, 데이터를 보낼 때 한 번에 1비트씩 차례대로 주고받는 통신 방식. TX (Transmit - 송신) / RX (Receive - 수신) 포트로 이루어지고 아두이노 메가는 총 4쌍의 시리얼 통신 포트가 있음.

# 1. Arduino Mega 알아보기

•SPI 통신 핀 : 50(MSIO), 51(MOSI), 52(SCK), 53(SS)

- MISO (Master In Slave Out): 슬레이브가 마스터(아두이노)에게 데이터를 보내는(아두이노가 받는) 선
- MOSI (Master Out Slave In): 마스터(아두이노)가 슬레이브(센서/부품)에게 데이터를 보내는 선
- SCK (Serial Clock): I2C의 SCL처럼 데이터 타이밍을 맞춰주는 클럭 선
- SS(Slave Select): **타깃 선택** 선

•I2C 통신 핀 : 20(SDA), 21(SCL) : 선이 공통으로 묶여 있기 때문에, 아두이노가 데이터를 보낼 때 맨 앞에 주소를 붙여서 전송.

- SDA(Data): 실제 데이터가 이동하는 선
- SCL(Clock): 클럭 선

## 2. NeRF

NeRF(Neural Radiance Fields, 신경 복사장)는 여러 각도에서 찍은 2D 이미지들을 바탕으로, 3D 공간/물체를 컴퓨터 그래픽스로 정밀하게 재생성해내는 인공지능 기술

### 1. NeRF는 어떻게 작동하는가?

기존의 3D 모델링은 물체의 표면에 점을 찍고(Point Cloud) 면을 만드는(Mesh) 방식을 사용. 반면 NeRF는 완전히 다른 접근 방식을 취한다.

•**공간을 함수로 표현:** 3D 공간의 특정 좌표  $(x, y, z)$ 와 바라보는 방향을 입력하면, AI(인공신경망)가 그 위치의 색상(RGB)과 밀도(Density, 불투명도)를 출력.

•**레이 트레이싱(Ray Tracing) 활용:** 카메라 렌즈에서 빛을 쏘아 보낸다고 가정하고, 그 빛이 지나가는 경로 상의 색상과 밀도를 인공신경망으로 계산한 뒤 모두 더해(렌더링) 최종 2D 이미지를 만든다.

•**학습 과정:** AI가 만든 이미지와 실제 촬영한 원본 사진을 비교하면서, 두 이미지가 똑같아질 때까지 인공신경망의 가중치(Weight)를 학습시킨다.

## 2. NeRF

### 2. NeRF의 장점

- **극도의 실사주의(Photorealism):** 유리잔의 복잡한 굴절, 금속 표면의 반사광, 반투명한 물체, 머리카락처럼 정밀한 디테일을 기존 3D 그래픽스보다 훨씬 현실감 있게 표현.
- **간편한 소스:** 고가의 3D 스캐너 없이, 스마트폰으로 동영상을 찍거나 여러 장의 사진을 촬영하는 것만으로도 3D 공간을 구축할 수 있다.

### 3. 핵심 기술 1: 위치 인코딩 (Positional Encoding)

단순히  $(x, y, z)$  좌표 숫자를 그대로 MLP(NeRF의 네트워크 구조)에 넣으면, 심층 신경망의 특성상 고주파(High-frequency) 정보(날카로운 경계면, 세밀한 질감 등)를 잘 학습하지 못하고 이미지가 뭉개지는 현상이 발생. 이를 해결하기 위해 NeRF는 입력 좌표를 고주파의 사인(sin)과 코사인(cos) 함수를 이용해 고차원 공간으로 매핑하는 **Positional Encoding**을 적용.

$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \sin(2^1 \pi p), \cos(2^1 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p))$$

## 2. NeRF

### 3. 핵심 기술 2: 볼륨 렌더링 (Volume Rendering)

카메라 렌즈 ( 원점  $O$  ) 에서 픽셀을 향해 가상의 빛(Ray)을 쏜다고 하자. 이 Ray 위의 임의의 점은 시간(거리)  $t$ 에 따라  $\mathbf{r}(t) = O + t \mathbf{d}$ 로 표현된다. 이 선을 따라 가며 색상들을 누적하여 최종 픽셀 색상  $C(\mathbf{r})$ 을 계산한다.

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \cdot \sigma(\mathbf{r}(t)) \cdot \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt$$

$T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$ : 투과율(Transmittance). 빛이  $t_n$ 부터  $t$ 까지 오면서 다른 물체에 가려지지 않고 살아남았을 확률. 앞부분에 불투명한 물체가 있으면  $T(t)$ 는 0에 수렴하여 뒤쪽 물체는 보이지 않게 된다.

$\sigma(\mathbf{r}(t))$ : 해당 지점의 밀도.

$\mathbf{c}(\mathbf{r}(t), \mathbf{d})$ : 해당 지점에서 시선 방향으로 방출되는 색상.

# 3. MVS

MVS(Multi-View Stereo, 다중 시점 스테레오)는 여러 각도에서 촬영한 여러 장의 2D 디지털 이미지들을 분석하여, 물체나 공간의 3D 기하학적 구조(형태)를 복원해내는 기술. 인간이 양쪽 눈(Stereo)의 시차를 이용해 입체감을 느끼는 원리를 컴퓨터로 확장하여, 수십~수백 장의 카메라 시점(Multi-View)에서 물체의 3D 좌표를 계산해내는 기술

## 1. MVS의 핵심 파이프라인 (작동 원리)

### ① 이미지 특징점 매칭 및 카메라 포즈 추정 (SfM 단계)

입력된 이미지들에서 특징점(SIFT 등)을 찾아 서로 매칭. 이를 통해 카메라의 행렬(Pose)을 역산해내고, 대략적인 물체의 뼈대(Sparse Point Cloud)를 만든다.

### ② 밀집 매칭 (Dense Matching - MVS의 핵심)

모든 픽셀의 3D 좌표를 정밀하게 채워 넣을 차례.

- 에피폴라 기하학(Epipolar Geometry): 한 이미지의 특정 픽셀이 다른 이미지에서는 어느 선(Line) 위에 위치해야 하는지 수학적으로 제한.

- 포토메트릭 일관성(Photometric Consistency): 그 선을 따라가며 색상과 패턴이 가장 유사한 픽셀을 찾는다. 똑같은 지점을 가리키는 픽셀 쌍을 찾으면, 삼각측량법(Triangulation)을 이용해 해당 픽셀의 정확한 깊이(Depth, 거리)를 계산.

### ③ 3D 구조물 생성

모든 픽셀의 깊이 정보가 모여 촘촘한 점구름(Dense Point Cloud)이 되고, 이 점들을 삼각형 껍데기로 이어 붙이면 우리가 흔히 보는 3D 그래픽 데이터인 메쉬(Mesh)와 텍스처(Texture)가 완성된다.



# 4 . 3D Gaussian Splatting (3DGS)

3D Gaussian Splatting (3DGS)은 사진을 주면, AI가 공간에 수백만 개의 투명한 물방울(가우시안)을 뿌려서 실제와 똑같은 3D 가상 공간을 실시간으로 만들어내는 기술

## 1. NeRF와의 결정적인 차이점: 함수 vs 점

- NeRF**: 공간을 하나의 수학적 함수(MLP 신경망)로 채움. 화면에 이미지 한 장을 그리려면 가상의 레이(Ray)를 쏘고 그 선 위의 수백 개 지점에서 신경망 연산을 일일이 실행해야 하므로 연산량이 매우 많음.
- 3DGS**: 공간에 수백만 개의 수치화된 3D 가우시안 타원체(Gaussian)를 직접 뿌림. 화면을 그릴 때는 이 타원체들을 카메라 화면에 픽셀 단위로 빠르게 투영(Splatting)하기 때문에 연산 속도가 압도적으로 빠르다.

# 4. 3D Gaussian Splatting (3DGS)

## 2. 3D 가우시안(Gaussian)이란 무엇인가?

3DGS에서 공간을 구성하는 최소 단위는 점(Point)이 아니라, 중심부에서 외각으로 갈수록 부드럽게 흐려지는 물방울 형태의 **3D 가우시안 타원체**이다. 하나의 가우시안은 다음과 같은 물리적 수학 정보를 가진다.

- 위치 (Position): 3D 공간 상의 중심 좌표 ( $x, y, z$ )
- 공분산 (Covariance): 타원체의 크기와 형태, 회전 방향을 결정하는 3D 타원체 행렬 (크기 조절을 위한 Scaling 행렬과 방향 조절을 위한 Rotation 행렬로 분해하여 표기)
- 불투명도 (Opacity): 이 물방울이 얼마나 불투명한지 (0~1 사이의 값)
- 색상 (Color): 시점(바라보는 각도)에 따라 다르게 보일 수 있도록 구면 조화 함수(Spherical Harmonics, SH) 계수로 색상을 표현.

# 4. 3D Gaussian Splatting (3DGS)

## 3. 3DGS의 파이프라인 (동작 매커니즘)

### ① 초기화 (Initialization)

먼저 전통적인 SfM을 돌려 카메라 위치와 대략적인 희소 점구름(Sparse Point Cloud)을 얻는다. 이 점구름 위치를 중심으로 3D 가우시안 타원체를 생성한다.

### ② 스플래팅 및 래스터화 (Splatting & Rasterization)

- Splatting**: 3D 가우시안들을 현재 카메라 렌즈 시점의 2D 화면상에 납작한 2D 가우시안(타원형 물방울)으로 투영한다.

- Tile-based Rasterizer**: 화면을 16 X 16 크기의 타일(Tile) 단위로 쪼갬다. 그리고 각 타일에 겹치는 2D 가우시안들을 카메라와의 거리(Depth) 순으로 정렬한다.

- Alpha Blending**: 정렬된 가우시안들을 맨 앞부터 순서대로 겹쳐 픽셀의 최종 색상을 계산한다. GPU 파이프라인에 최적화된 하드웨어 친화적 방식(Rasterization)을 쓰기 때문에 렌더링 속도가 빠르다.

## 4. 3D Gaussian Splatting (3DGS)

### ③ 적응형 밀도 제어 (Adaptive Density Control)

렌더링된 결과와 실제 사진의 오차를 계산하여 각 가우시안의 위치, 크기, 색상, 불투명도를 최적화(Adam Optimizer)한다. 이때 구조가 복잡한 곳이나 빈 공간을 채우기 위해 가우시안을 스스로 제어합니다.

- **Cloning (복제):** 작은 가우시안이 있는 곳에 디테일이 부족하면 동일한 가우시안을 하나 더 복제하여 채운다.
- **Splitting (분할):** 하나의 거대한 가우시안이 너무 넓은 범위를 차지하고 있으면, 이를 더 작은 가우시안 여러 개로 쪼개고 크기를 줄여 정밀하게 조절한다.

# 4. 3D Gaussian Splatting (3DGS)

## 4. 3DGS 장점

- 1. 실시간성:** 3DGS는 순수 가우시안 데이터 상태로 스마트폰이나 웹 브라우저에서도 60~100+ FPS로 매끄럽게 구동된다.
- 2. 빠른 학습 속도:** 수 시간~수 일이 걸리던 NeRF와 달리, 3DGS는 고성능 GPU 기준 **20~30분 만에 학습을 완료**하고 고품질의 3D 공간을 만들어낸다.
- 3. 수정 및 편집의 용이성:** NeRF는 전체 공간이 신경망 가중치(Weight)로 꼬여 있어서 특정 물체만 지우거나 옮기기가 불가능하다. 반면 3DGS는 가우시안이라는 개별 입자들의 모임이므로, **원치 않는 부분의 가우시안 무리를 지우거나 크기를 바꾸는 물리적 편집이 매우 직관적**이다.

# 4. 3D Gaussian Splatting (3DGS)

## 5. 3DGS의 한계점

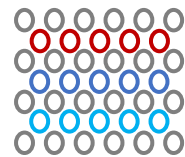
- 방대한 저장 용량:** 수백만 개의 가우시안 하나하나마다 위치, 크기, 회전, SH 계수 등을 다 저장해야 하므로, 부피가 큰 장면의 경우 파일 용량이 수백 MB에서 수 GB에 달해 웹 스트리밍이나 모바일 저장에 부담이 된다.
- 메쉬(Mesh) 변환의 어려움:** 가우시안은 부드러운 물방울 무리이기 때문에, 게임 엔진(Unity, Unreal)이나 3D 프린터에서 쓰는 딱딱한 다각형 표면(Mesh) 데이터로 깨끗하게 변환하는 것이 어렵다. 흔히 외각이 지저분해지거나 흐릿해지는 현상이 발생한다.

## 5. 최종 비교

| 비교 항목       | MVS (1세대: 전통 기하학)                | NeRF (2세대: AI 함수)             | 3DGS (3세대: AI 입자)                    |
|-------------|----------------------------------|-------------------------------|--------------------------------------|
| 공간을 정의하는 방식 | 딱딱한 표면 껍데기<br>(점구름, 다각형 메쉬)      | 연속적인 수학 공식<br>(인공신경망 MLP)     | 부드러운 물방울 무리<br>(3D 가우시안 타원체)         |
| 렌더링 속도      | 아주 빠름 (기존 그래픽 파이프라인 활용)          | 치명적으로 느림<br>(픽셀당 수백 번 함수 연산)  | 압도적인 실시간 (100+ FPS)<br>(GPU 고속 래스터화) |
| 학습/연산 속도    | 빠름 (수학적 삼각측량 기반)                 | 아주 느림 (수 시간~수 일)              | 매우 빠름 (20~30분)                       |
| 최대 장점       | 정확한 수치와 치수<br>(CAD/3D 프린팅 변환 용이) | 극도의 실사주의<br>(투명체, 복잡한 반사광 표현) | 실사 퀄리티 + 실시간 구동<br>(물체 단위 개별 편집도 가능) |
| 치명적인 단점     | 밋밋한 벽이나 유리를 인식 못 해 구멍이 뚫림        | 연산량이 너무 무거워 모바일/웹 구동이 어려움     | 데이터 파일 용량이 큼 (수백 MB ~ 수 GB)          |
| 주요 활용 분야    | 구글 어스 3D 지도, 건축 측량, 문화재 디지털 트윈   | 영화 VFX, 고품질 3D 그래픽스 연구        | 자율주행 시뮬레이터, VR/AR, 메타버스 가상 공간 복원     |

# Q & A

## Thank you for your attention



Architecture and Compiler  
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU  
ACE Lab. (hw051656@gmail.com)